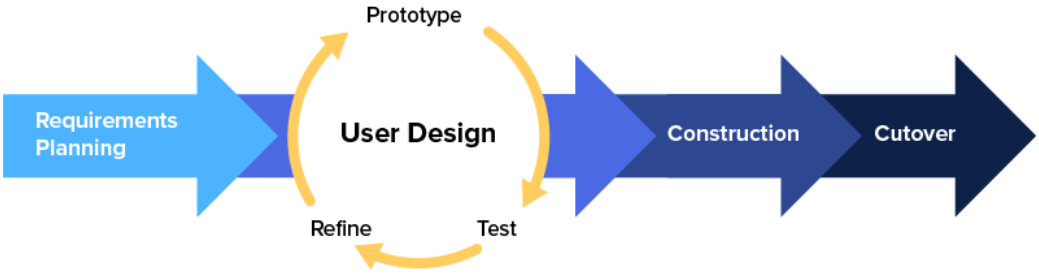
BAB III
METODE PENELITIAN

3.1 Gambaran Umum



Gambar 3.1 Rapid Application Development (RAD)

Dalam merancang dan membangun Sistem Informasi Absensi Karyawan Berbasis Web ini, metodologi yang digunakan adalah ***Rapid Application Development (RAD)***. Pemilihan metode RAD didasarkan pada karakteristiknya yang adaptif, tangkas, serta berorientasi pada siklus pengembangan yang singkat melalui pelibatan pengguna secara intensif. Berdasarkan skema alur pengembangan yang dirancang, logika penjelasan dari tahapan-tahapan RAD yang disesuaikan ke dalam proyek sistem absensi pengenalan wajah (*face recognition*) ini dijabarkan sebagai berikut:

3.1.1 Requirement Planning (Perencanaan Kebutuhan)

Tahap awal dari siklus RAD difokuskan pada pendefinisian ruang lingkup, tujuan, serta prasyarat fungsionalitas utama dari sistem absensi yang akan dibangun. Logika pada tahap ini meliputi:

- **Analisis Masalah Manual**: Mengidentifikasi kelemahan sistem pencatatan kehadiran konvensional, seperti adanya antrean panjang, ketidakpraktisan perekapan, hingga celah manipulasi data (titip absen).
- **Penentuan Kebutuhan Solusi**: Merencanakan kebutuhan sistem otomatis berbasis *web* yang mampu mengenali identitas biometrik wajah karyawan secara langsung (*real-time*) tanpa kontak fisik.
- **Spesifikasi Komponen Teknis**: Menetapkan instrumen utama pembangunan sistem, yaitu bahasa pemrograman **Python** dengan *framework* **Flask** untuk sisi *backend server*, serta pustaka **dlib** dan **OpenCV** untuk mengoordinasikan pemrosesan pengenalan wajah.

3.1.2 RAD Design Workshop (Desain dan Konstruksi Prototipe)

Tahap ini merupakan fase krusial dalam RAD yang bekerja secara iteratif (berulang) untuk mengubah hasil perencanaan kebutuhan menjadi prototipe aplikasi yang fungsional. Alur kerja logis pada tahap ini dibagi menjadi tiga sub-proses utama:

- **Sistem Analis & Desain (UML/ERD)**: Merancang alur logis program menggunakan diagram pemodelan visual seperti *use case diagram* untuk hak akses pengguna (admin dan karyawan), *activity diagram* untuk pemodelan aktivitas absensi depan kamera, serta *Entity Relationship Diagram* (ERD) untuk penataan basis data relasional karyawan.
- **Konstruksi Pengolahan Citra & Model AI**:
    - Mengintegrasikan fungsionalitas kamera tangkap (*webcam*) melalui OpenCV untuk melacak keberadaan wajah karyawan pada aliran video.
    - Memanfaatkan fungsi *deep metric learning* dari *library* dlib untuk mengekstrak titik struktural wajah (*facial landmarks*) karyawan menjadi daftar matriks unik vektor 128 dimensi (*128-d vector*) sebagai data *encoding* wajah.

- Menghubungkan data *encoding* tersebut ke dalam database agar siap dicocokkan secara aktual menggunakan algoritma klasifikasi ketetanggaan terdekat (*K-Nearest Neighbor*) saat karyawan melakukan pemindaian kehadiran.

- **Uji Coba & Demonstrasi Prototipe (User Review)**:

  Menjalankan prototipe antarmuka Flask di lingkungan peladen lokal (*localhost*) untuk didemonstrasikan kepada dosen pembimbing. Jika terdapat kekurangan visual atau kegagalan pembacaan sudut wajah, sistem akan langsung kembali ke tahapan desain untuk direvisi secara cepat tanpa harus merombak fondasi program dari awal.

3.1.3 *Implementation / Cutover* (Implementasi Sistem Final)

Fase akhir dari metodologi RAD yang dilakukan apabila seluruh fungsi prototipe aplikasi absensi pengenalan wajah telah dinyatakan valid, bebas dari kerentanan (*bug*), serta lolos tahap pengujian fungsional. Logika pelaksanaan pada tahap ini meliputi:

- **Penerapan di Jaringan Lokal (LAN)**: Mengunggah kode program final ke perangkat server utama agar aplikasi absensi berbasis Flask tersebut dapat diakses secara simultan oleh komputer karyawan klien melalui jaringan LAN tempat bekerja.
- **Migrasi Data Aktual**: Memindahkan berkas foto wajah dan identitas seluruh karyawan asli ke dalam folder dataset peladen untuk dieksekusi oleh modul *training data* dlib secara permanen.
- **Pemberlakuan Sistem Baru**: Menghentikan operasional pencatatan kehadiran manual dan sepenuhnya mengalihkan pencatatan presensi kedatangan maupun kepulangan karyawan ke dalam sistem informasi absensi otomatis berbasis *face recognition* yang baru secara aktual (*real-time*).

3.2 Algoritma Pemrosesan Citra dan Pengenalan Wajah

Proses pengenalan wajah pada sistem absensi ini didasarkan pada tiga tahapan komputasi utama yang bekerja di latar belakang peladen Flask, yaitu deteksi lokasi wajah (*face detection*), ekstraksi fitur geometris (*face encoding*), dan kalkulasi pencocokan identitas (*face comparing*). Pada tahap awal, OpenCV melalui fungsi VideoCapture melakukan akuisisi aliran bingkai video dari perangkat kamera secara *real-time*. Bingkai gambar yang awalnya berada dalam ruang warna BGR (*Blue, Green, Red*) dikonversi menjadi ruang warna RGB (*Red, Green, Blue*) agar dapat diproses oleh library dlib menggunakan algoritma *Histogram of Oriented Gradients* (HOG). Algoritma HOG ini bekerja dengan memetakan distribusi gradien orientasi intensitas cahaya untuk melacak dan mengunci posisi koordinat kotak wajah objek di dalam matriks citra.

Setelah lokasi wajah berhasil dikunci, dlib menerapkan model *shape predictor* 68 titik penanda wajah (*facial landmarks*) untuk mendeteksi struktur fisik esensial seperti posisi mata, alis, hidung, bibir, dan garis rahang. Titik-titik koordinat spasial tersebut kemudian diumpankan ke dalam arsitektur jaringan saraf tiruan deep learning *ResNet-34* milik dlib. Jaringan ini bertugas mengekstraksi karakteristik visual unik tersebut menjadi sebuah representasi digital berupa vektor metrik berdimensi 128 yang disebut sebagai *face embedding*. Vektor inilah yang merepresentasikan identitas biometrik digital masing-masing karyawan secara spesifik dan disimpan ke dalam database MySQL dalam tipe data *longtext*.

Tahapan dari sistem ini adalah penentuan kecocokan identitas (*face comparing*) antara wajah baru yang tertangkap kamera dengan sampel data karyawan yang telah terdaftar di database. Sistem memvalidasi parameter kedekatan visual ini menggunakan perhitungan matematis Jarak Euclidean (*Euclidean Distance*). Secara teknis, Jarak Euclidean mengukur panjang garis lurus antara dua titik vektor berdimensi 128 di dalam ruang Euclidean untuk mengetahui tingkat kemiripan kedua

wajah tersebut. Rumus perhitungannya dinyatakan secara matematis melalui Persamaan berikut:

$$d(x,y) = \sqrt{\left\{ \sum_{\{i=1\}}^{\{n\}(x_i - y_i)^2} (x_i - y_i)^2 \right\}} \quad (1)$$

Di mana d(x, y) merepresentasikan nilai total Jarak Euclidean yang dihasilkan, sementara xi adalah nilai elemen vektor wajah masukan dari kamera pada dimensi ke-i, yi merupakan nilai elemen vektor sampel wajah karyawan di dalam database pada dimensi ke-i, dan n melambangkan batas total dimensi vektor fitur yang bernilai 128.

Di dalam skrip Flask, hasil perhitungan jarak d(x, y) akan dibandingkan dengan nilai ambang batas toleransi (*threshold*) yang telah ditetapkan secara global sebesar 0.55. Jika nilai d(x, y) berada di bawah atau sama dengan nilai *threshold*, sistem akan mengonfirmasi bahwa kedua wajah tersebut identik, lalu menarik data profil karyawan dan mencatat stempel waktu kehadiran ke basis data absensi. Sebaliknya, apabila nilai jarak komputasi melebihi batas *threshold*, wajah objek visual tersebut akan dinyatakan tidak dikenal (*unknown*) untuk mencegah terjadinya kesalahan input data kehadiran pekerja.

3.2.1 Penyusunan Alur Rinci Algoritma ekstraksi Citra

Berikut tahapan ekstraksi citra yang di lakukan oleh sistem face recognition :

1. **Akuisisi citra wajah:** Menggunakan OpenCV (cv2.VideoCapture), lalu konversi format warna dari **BGR** => **RGB**.
2. **Deteksi wajah:** Menggunakan algoritma **HOG (Histogram of Oriented Gradients)** dari pustaka dlib.
3. **Deteksi landmark & alignment:** Menggunakan model shape_predictor_68_face_landmarks.dat untuk memetakan 68 titik koordinat spasial (mata, hidung, bibir, rahang) agar posisi wajah disejajarkan (*alignment*) secara tegak.
4. **Ekstraksi fitur dengan CNN/dlib:** Citra wajah yang telah disejajarkan diubah menjadi representasi **128 nilai numerik (*128-d vector face embedding*)** berbasis ResNet-34.
5. **Registrasi wajah:** Alur: *Input data karyawan => capture 5 sudut wajah => deteksi =>alignment => encoding => simpan string encoding ke kolom face_encoding MySQL*.
6. **Pencocokan wajah & Penentuan identitas:** Menggunakan rumus **Jarak Euclidean** dengan *threshold* **0,55**. Tambahkan landasan bahwa *threshold* 0,55 merupakan nilai batas standar optimal dari dlib yang divalidasi pada hasil pengujian Bab 4 (menghasilkan akurasi 100% tanpa salah identifikasi).
7. **Integrasi sistem absensi:** Alur: *Identitas cocok => validasi status tanggal hari ini => catat timestamp =>tampilkan log di tabel antarmuka*.
8. **Penanganan wajah tidak dikenal:** Jika jarak > 0,55$, kembalikan respon status: failed ("Wajah tidak dikenal!") tanpa menyimpan ke database.

3.2.2 Mekanisme Kalkulasi Durasi Jam Kerja Otomatis

Selain memvalidasi kecocokan biometrik wajah, sistem dilengkapi dengan logika *back-end* peladen Flask untuk mengalkulasi total durasi jam kerja karyawan secara otomatis setiap kali proses pemindaian presensi kepulangan (*scan keluar*) berhasil dilakukan.

Perhitungan durasi kerja dilakukan menggunakan fungsi hitung_durasi(jam_masuk, jam_keluar). Logika matematis dan alur komputasi fungsi tersebut dijabarkan sebagai berikut:

1. **Transformasi Format Waktu:**

   Data jam_masuk yang diambil dari rekaman basis data hari bersangkutan dan jam_keluar dari stempel waktu (*timestamp*) server saat ini dikonversi dari tipe teks menjadi objek representasi waktu datetime Python dengan pola format $\%H{:}\%M{:}\%S$.

2. **Kalkulasi Selisih Waktu (Δt):**

   Selisih total waktu dihitung dengan mengurangkan objek waktu kepulangan terhadap kedatangan:

$$\Delta t \; = \; t_{\{\backslash text\{keluar\}\}} - \; t_{\{\backslash text\{masuk\}\}}$$

3. **Konversi ke Satuan Detik:**

   Objek perbedaan waktu (timedelta) ditransformasikan ke dalam nilai bilangan bulat total detik $\left(S_{\{\backslash text\{total\}\}}\right)$ melalui persamaan:

$$S_{\{\backslash text\{total\}\}} = \backslash text\{int\}(\Delta t.\backslash text\{total_seconds\})$$

4. **Dekomposisi Satuan Jam, Menit, dan Detik:**

   Nilai $S_{\{\backslash text\{total\}\}}$ kemudian didekomposisi ke dalam besaran jam (J), menit (M), dan detik (D) menggunakan operasi pembagian bilangan bulat (*floor division*) dan modulus:

$$\mathrm{J} \; = \; \left\lfloor \frac{\mathrm{S}_{\{\backslash \mathrm{text}\{\mathrm{total}\}\}}}{} 3600 \right\rfloor$$

$$M = \left\lfloor \frac{\left(S_{\{\backslash text\{total\}\}\backslash pmod\{3600\}}\right)}{} 60 \right\rfloor$$

$$D = S_{\{\backslash text\{total\}\}\backslash pmod\{60\}}$$

5. **Pemformatan Nilai Keluaran (*String Output*):**
    - Jika $J = 0 dan M = 0, maka hasil diformat menjadi: "{D} detik".
    - Jika $J = 0 dan M > 0, maka hasil diformat menjadi: "{M} menit {D} detik".
    - Jika J > 0, maka hasil diformat menjadi: "{J} jam {M} menit".

Nilai teks hasil pemformatan tersebut selanjutnya disimpan secara permanen ke dalam kolom total_kerja pada tabel absensi di basis data MySQL.

3.3 Analisis Kebutuhan Sistem (*Requirement Analysis*)

Analisis kebutuhan sistem merupakan tahap identifikasi terhadap seluruh komponen pendukung dan spesifikasi teknis yang diperlukan untuk membangun aplikasi absensi karyawan berbasis *face recognition*. Tahapan ini mencakup spesifikasi perangkat keras, perangkat lunak, serta analisis kebutuhan fungsional dan non-fungsional sistem.

1. Kebutuhan Perangkat Keras (*Hardware Requirements*)

Perangkat keras yang digunakan dalam penelitian ini adalah sebagai berikut:
Prosesor:

- Intel Core i5
- RAM: 8 GB
- Penyimpanan: SSD 512 GB
- Sistem Operasi: Windows 11

2. Kebutuhan Perangkat Lunak (*Software Requirements*)

Perangkat Lunak: Peneliatian ini menggunakan perangkat sebagai berikut :

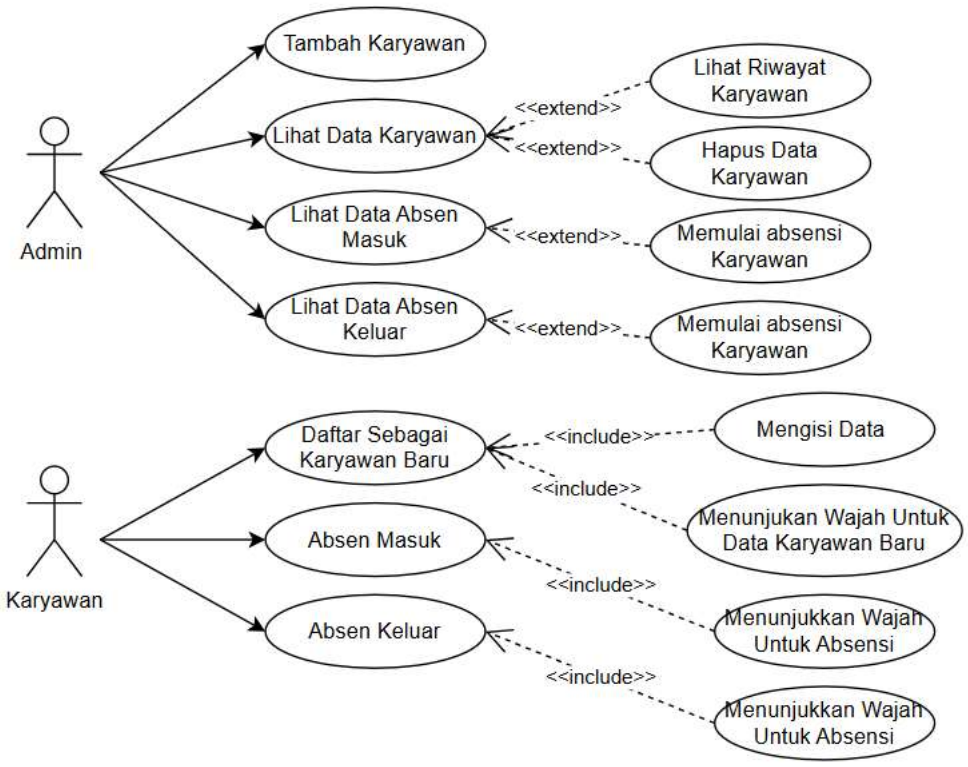
- XAMPP sebagai server lokal untuk pengembangan PHP dan MySQL.
- Visual Studio Code sebagai teks editor.
- Web browser (Google Chrome dan Micrososft Edge) untuk uji coba antarmuka
- Figma untuk desain antarmuka pengguna (UI Design).

3.4. Pemodelan dan Perancangan Sistem

Pemodelan dan perancangan sistem merupakan tahapan krusial dalam rekayasa perangkat lunak yang berfungsi sebagai jembatan untuk menerjemahkan spesifikasi kebutuhan fungsional ke dalam rancangan arsitektur teknis terstruktur.

3.4.1 *Use Case Diagram*

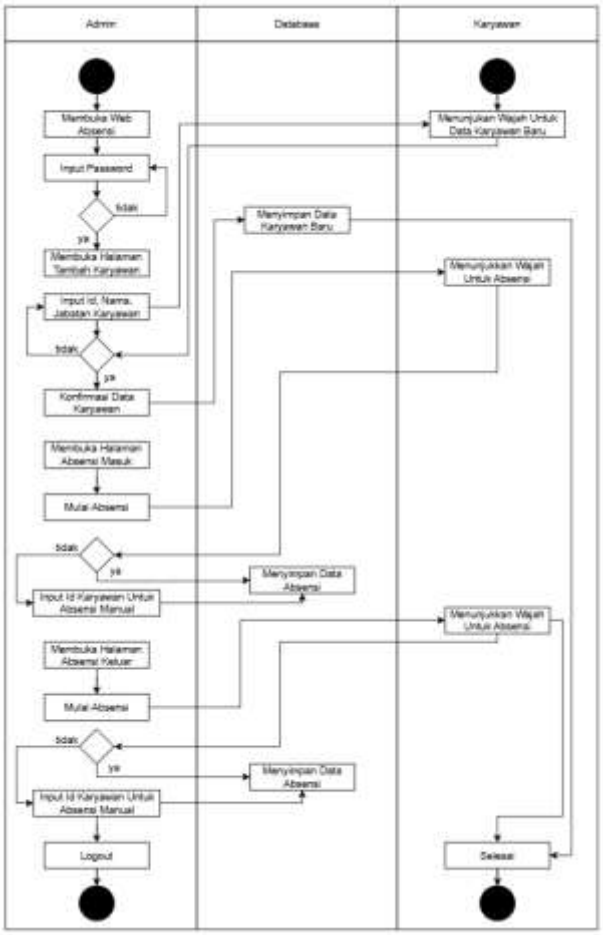
Use case diagram merupakan sebuah diagram UML yang bersifat statis untuk memperlihatkan himpunan kasus penggunaan (*use case*) serta aktor-aktor yang terlibat di dalam lingkungan sistem. Diagram ini mengemban dua fungsi utama dalam fase perancangan, yaitu mendefinisikan fitur atau layanan apa saja yang wajib disediakan oleh sistem sekaligus menyatakan sifat dan perilaku sistem tersebut dari sudut pandang pengguna (*user*). Berdasarkan rancangan *use case diagram* yang diusulkan pada Gambar 3.2, batas sistem aplikasi (*system boundary*) memisahkan fungsionalitas sistem absensi ke dalam hak akses dua aktor utama, yaitu Admin dan Karyawan.



Gambar 3.2 Use case Diagram

3.4.2 *Activity Diagram*

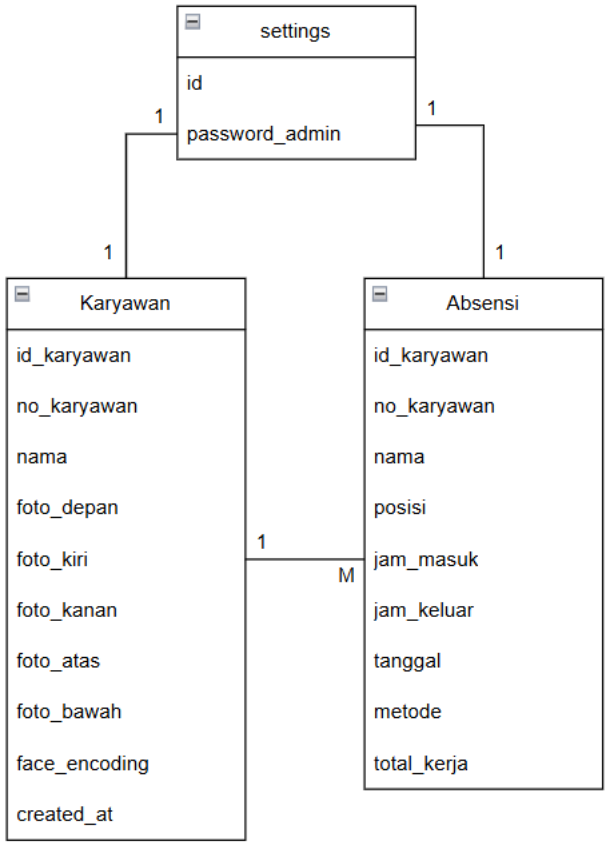
Activity diagram atau diagram aktivitas berfungsi untuk menggambarkan aliran kerja (*workflow*) dari proses bisnis, langkah-langkah fungsional, serta logika perilaku objek di dalam sebuah proses secara grafis. Fokus utama dari diagram ini adalah memodelkan urutan aktivitas dinamis sistem dan memperlihatkan bagaimana interaksi terjadi di antara berbagai entitas yang terlibat. Berdasarkan rancangan yang diusulkan pada Gambar 3.3, diagram aktivitas ini dibagi ke dalam tiga jalur renang (*swimlane*) utama yang merepresentasikan pembagian peran secara terstruktur, yaitu Admin, Database, dan Karyawan.



Gambar 3.3 Activity Diagram

3.4.3 *Class Diagram*

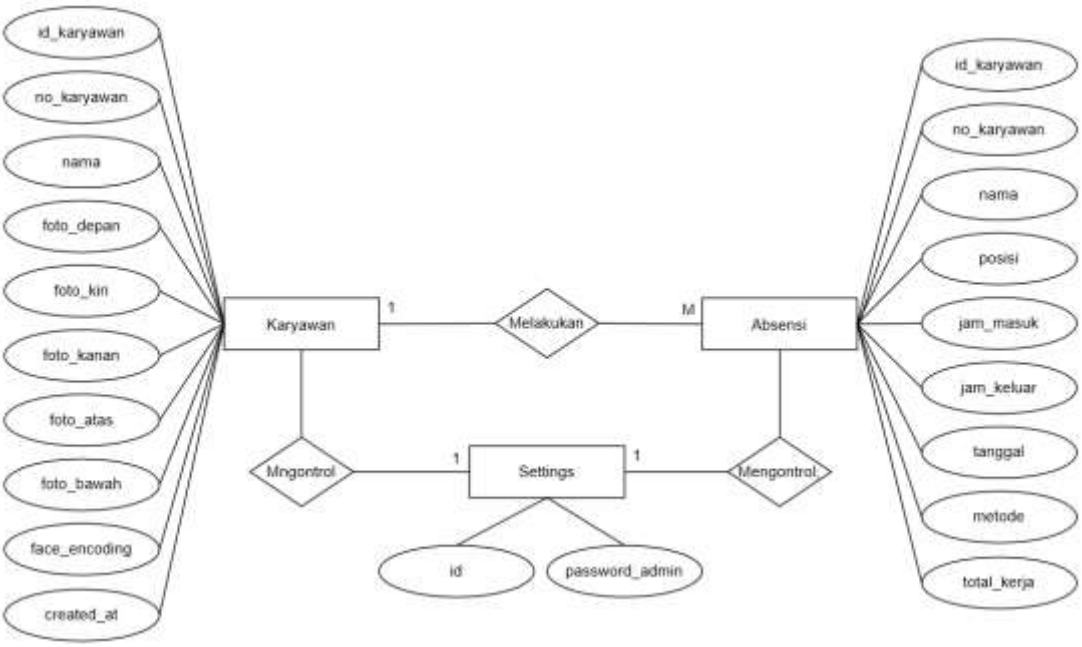
Class diagram atau diagram kelas merupakan pemodelan statis berorientasi objek yang menunjukkan struktur sistem dengan mendefinisikan kelas-kelas yang akan dibuat untuk membangun perangkat lunak. Diagram ini memetakan kelompok objek yang memiliki properti (atribut), perilaku, serta relasi yang sama, sehingga mampu memberikan pandangan global atas sebuah arsitektur sistem informasi. Berdasarkan rancangan kelas pemrograman yang diusulkan pada Gambar 3.4, struktur logika aplikasi absensi berbasis Flask dan dlib ini disusun oleh tiga kelas modular utama, yaitu settings, Karyawan, dan Absensi.



Gambar 3.4 Class Diagram

3.4.4 *Entity Relationship Diagram* (ERD)

Entity Relationship Diagram (ERD) merupakan sebuah diagram yang digunakan untuk memodelkan struktur data serta hubungan antar-objek data secara logis dan konseptual. Sebagai rancangan basis data relasional, ERD memetakan komponen entitas, atribut unik penyusun, hingga derajat kardinalitas yang mengikat hubungan tersebut berdasarkan aturan bisnis yang berlaku di dalam sistem. Berdasarkan skema logis basis data yang diusulkan pada Gambar 3.5, arsitektur data pada aplikasi absensi *face recognition* ini dibangun oleh tiga entitas utama, yaitu settings, Karyawan, dan Absensi.



Gambar 3.5 Entity Relationship Diagram (ERD)

3.5 Perancangan Basis Data

Basis data yang digunakan pada sistem absensi *Coffee ID* dirancang menggunakan MySQL dengan nama database skripsi_absensi. Database ini terdiri dari 3 tabel utama, yaitu tabel karyawan, tabel absensi, dan tabel settings. Berikut adalah struktur dari masing-masing tabel:

3.5.1 Perancangan Basis Data Karyawan

Tabel Karyawan pada Tabel 3.1 digunakan untuk menyimpan data induk profil setiap karyawan beserta hasil ekstraksi karakteristik wajah (*face encoding*) yang digunakan dalam pencocokan biometrik.

Tabel 3.1 Perancangan Basis Data Absensi

Nama Kolom	**Tipe Data**	**Keterangan**
id,	Int(11),	ID karyawan *(Auto Increment, Primary Key)*,
no_karyawan,	Varchar(50),	Nomor induk unik karyawan,
nama,	Varchar(100),	Nama lengkap karyawan,
posisi,	Varchar(50),	Posisi atau jabatan kerja karyawan,
foto_depan,	Varchar(255),	Jalur direktori (*path*) file foto tampak depan,
foto_kiri,	Varchar(255),	Jalur direktori (*path*) file foto tampak kiri,
foto_kanan,	Varchar(255),	Jalur direktori (*path*) file foto tampak kanan,
foto_atas,	Varchar(255),	Jalur direktori (*path*) file foto tampak atas,
foto_bawah,	Varchar(255),	Jalur direktori (*path*) file foto tampak bawah,
face_encoding,	Longtext	Vektor matriks berisi 128 nilai angka unik wajah

3.5.2 Perancangan Basis Data Absensi

Tabel Absensi Pada Tabel 3.2 berfungsi untuk mencatat log riwayat kehadiran harian karyawan secara *real-time*, baik melalui pemindaian wajah maupun input otorisasi manual, serta menyimpan kalkulasi durasi kerja otomatis.

Tabel 3.2 Perancangan Basis Data Absensi

Nama Kolom	**Tipe Data**	**Keterangan**
id,	Int(11),	ID log absensi *(Auto Increment, Primary Key)*,
no_karyawan,	Varchar(50),	Nomor induk karyawan *(Foreign Key)*,
nama,	Varchar(100),	Nama lengkap karyawan saat melakukan absensi,
posisi,	Varchar(50),	Posisi kerja karyawan saat melakukan absensi,
tanggal,	Date,	Tanggal pelaksanaan absensi (YYYY-MM-DD),
jam_masuk,	Time,	Penanda waktu (*timestamp*) server jam masuk,
jam_keluar,	Time,	Penanda waktu (*timestamp*) server jam keluar *(Null jika belum keluar)*,
total_kerja,	Varchar(100),	Kalkulasi durasi jam kerja harian karyawan,
metode,	Varchar(50)	Metode verifikasi absensi (*Face Recognition* / *Manual*)

3.5.3 Perancangan Basis Data Settings

Tabel Settings pada Tabel 3.3 digunakan untuk menyimpan parameter konfigurasi sistem global, seperti enkripsi kata sandi yang dipegang oleh *owner* atau administrator untuk keperluan otorisasi khusus.

Tabel 3.3 Perancangan Basis Data Settings

Nama Kolom	**Tipe Data**	**Keterangan**
id,	Int(11),	ID pengaturan *(Primary Key)*,
password_admin,	Varchar(255)	Kata sandi owner untuk otorisasi log website, log absensi masuk dan keluar manual

3.6 Tahapan Perancangan Antarmuka

Perancangan antarmuka pengguna bertujuan untuk memberikan gambaran logis mengenai tata letak, penempatan komponen input-output, serta navigasi visual pada website sebelum masuk ke tahap pengodean skrip antarmuka. Struktur perancangan antarmuka pada website absensi ini dibagi menjadi enam halaman operasional utama untuk admin dan karyawan.

3.6.1 Perancangan Halaman Login

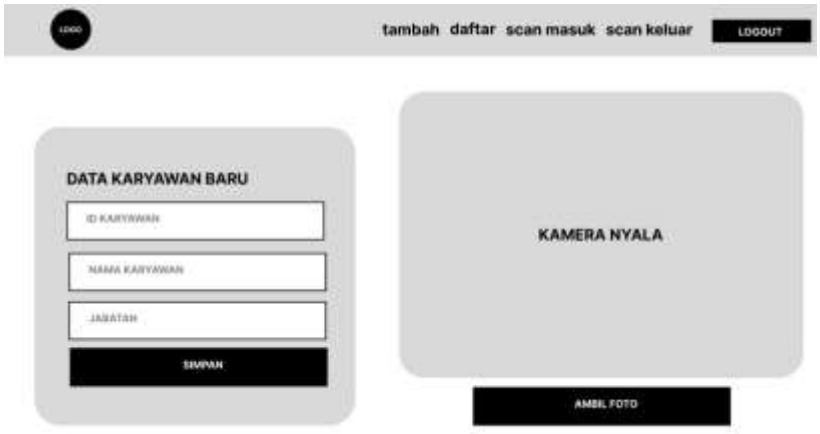
Halaman Login Admin pada Gambar 3.6. merupakan pintu gerbang utama untuk mengamankan hak akses pengelola sistem *back-end*. Komponen penyusun antarmuka halaman ini dimana admin di awali dengan input password yang sudah di setting passwordnya yang di simpan pada tabel settings pada database.



Gambar 3.6 Halaman Login

3.6.2 Perancangan Halaman Tambah Karyawan

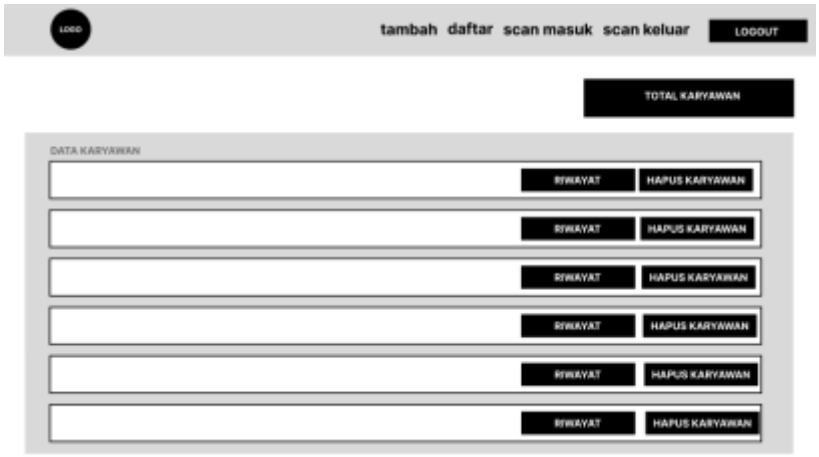
Halaman Tambah Karyawan pada Gambar 3.7 terdapat rancangan yang digunakan oleh Admin untuk mendaftarkan identitas profil serta sampel biometrik karyawan baru ke dalam server. Antarmuka halaman ini dilengkapi dengan *bar* navigasi di bagian atas yang menyediakan tautan menu tambah, daftar, scan masuk, scan keluar, serta tombol Logout untuk mempermudah Admin berpindah halaman kerja. Di bawah navigasi sebelah kiri, terdapat form data karyawan baru yang terdiri dari tiga kolom isian wajib yaitu Id Karyawan, Nama Karyawan dan Jabatan, yang diakhiri dengan tombol simpan. Sementara itu, di sisi kanan form terdapat sebuah panel kamera nyala yang menampilkan aliran video *real-time* dari kamera via OpenCV, lengkap dengan tombol ambil foto di bawahnya yang berfungsi untuk menangkap citra wajah karyawan baru.



Gambar 3.7 Halaman Tambah Karyawan

3.6.3 Perancangan Halaman Daftar Karyawan

Halaman Daftar Karyawan pada Gambar 3.8 terdapat rancangan yang berfungsi untuk menampilkan seluruh direktori data pekerja yang telah aktif terdaftar di dalam sistem basis data. Di sudut kanan atas halaman ini, terdapat sebuah tombol total karyawan yang berfungsi untuk memunculkan ringkasan jumlah total data pekerja secara aktual. Bagianhk utama halaman ini didominasi oleh tabel data karyawan yang menyajikan informasi identitas para pekerja secara teratur. Agar Admin dapat mengelola data dengan praktis, setiap baris informasi di dalam tabel tersebut telah dilengkapi dengan dua tombol aksi interaktif di ujung kanan, yaitu tombol riwayat untuk meninjau log kehadiran individu dan tombol hapus karyawan untuk menghapus profil pekerja dari database sistem.



Gambar 3.8 Halaman Daftar Karyawan

3.6.4 Perancangan Halaman Riwayat Karyawan

Halaman Riwayat Karyawan pada Gambar 3.9 terdapat rancangan yang menyediakan fungsi peninjauan mendalam terhadap rekapitulasi data kehadiran pekerja yang dipilih. Pada bagian kiri atas halaman, terdapat kotak data karyawan yang menampilkan profil singkat beserta foto wajah karyawan, yang diletakkan berdampingan dengan kotak riwayat kerja di sebelah kanannya. Tepat di bawah kedua kotak tersebut, dirancang sebuah kotak khusus untuk filter data karyawan yang berfungsi sebagai kolom pencarian atau penyortiran log kehadiran berdasarkan parameter tertentu. Hasil dari penyaringan tersebut akan langsung disajikan pada bagian paling bawah halaman di dalam kotak hasil filter data karyawan, yang menampilkan tabel rekap log data jam masuk, jam keluar, dan total jam kerja secara sistematis.



Gambar 3.9 Halaman Riwayat Karyawan

3.6.5 Perancangan Halaman Absensi Masuk

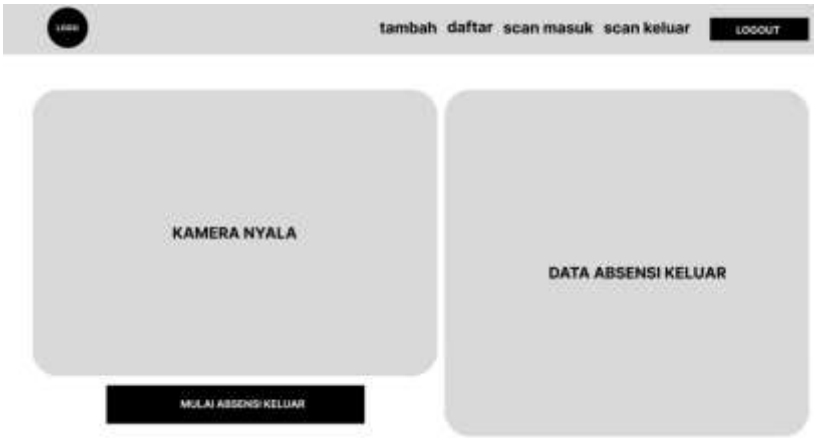
Halaman Absensi Masuk pada Gambar 3.10 terdapat rancangan yang menjadi antarmuka operasional harian bagi karyawan untuk melakukan pencatatan presensi kehadiran saat mulai bekerja. Tata letak halaman ini dibagi menjadi dua panel besar yang diletakkan berdampingan secara horizontal. Panel sebelah kiri berupa panel kamera nyala yang berfungsi menampilkan sorotan kamera aktif untuk mendeteksi wajah karyawan, lengkap dengan tombol mulai absensi masuk di bawahnya untuk mengaktifkan modul komputasi pengenalan wajah. Ketika wajah berhasil dikenali oleh sistem atau saat dilakukan input manual, data riwayat tersebut akan otomatis dimunculkan pada panel data absensi masuk yang berada di sisi kanan halaman.



Gambar 3.10 Halaman Absensi Masuk

3.6.6 Perancangan Halaman Absensi Keluar

Halaman Absensi Keluar pada Gambar 3.11 terdapat rancangan yang memiliki struktur tata letak serupa dengan halaman absensi masuk, namun dikonfigurasikan khusus untuk menangani presensi kepulangan kerja. Pada panel sebelah kiri, terdapat panel kamera nyala yang berfungsi menangkap umpan balik visual dari biometrik wajah karyawan, disertai dengan tombol mulai absesnsi keluar di bagian bawahnya untuk memicu proses pemindaian wajah. Begitu sistem berhasil mengenali wajah karyawan yang akan pulang, data log kepulangan tersebut akan langsung ditampilkan secara aktual pada panel data absensi keluar di sisi kanan halaman, yang kemudian memicu server Flask untuk memperbarui data dan mengalkulasi nilai total jam kerja karyawan.



Gambar 3.11 Halaman Absensi Keluar